

Deliverable 3: Database Implementation and Application

Project: Research Lab Manager

Members: Rohit Deshmukh, Joao Christiansen, Manisha Dwivedi

Course: CS 631 — Project Phase 3

1. Goals and Revisions

Goals for Phase 3

The primary goal of this phase was to move from logical design to a working system: provision a **database instance** with **realistic sample data**, and build a **menu-driven application** that connects to SQL Server and supports day-to-day lab operations and reporting. Concretely, this phase involved:

- Restoring and hosting the **Research Lab Manager** database on **Microsoft SQL Server** (containerized for consistency).
- Ensuring **sufficient tuples** in each table so that joins, aggregates, and reporting queries return meaningful results.
- Implementing **Java** application logic with **JDBC**, organized in a **DAO** (**LabManagerDAO**) and a **JavaFX** front end (**LabManagerController** + FXML) with a practical, tab-based workflow.
- Covering all **required menus and queries** from the Phase 3 specification: project/member management, equipment and usage tracking, and grant/publication analytics.

Revisions to the Relational Schema (from Deliverable 2)

Deliverable 2's DDL remains the conceptual foundation. For implementation, the live database and application align with that schema with these practical adjustments:

- **WorksOn:** The application models project staffing as (**MemberID, ProjectID**) assignments. Deliverable 2 also listed **Role** and **WeeklyHours** for richer work tracking; those columns may still exist in the physical database from the restored backup, but the current UI and DAO paths for add/update project members **do not set Role or WeeklyHours**, focusing on membership assignment only.

- **ResearchProjects:** New projects are created in the app with **Title** and **Status**. Other attributes from Phase 2 (e.g., start/end dates, planned duration, lead member) remain available at the database level where present in the backup but are **not all exposed** in the current CRUD dialogs.
- **LabMembers:** The UI manages **FirstName**, **LastName**, and **JoinDate**. **MiddleName** remains a valid column from the Phase 2 design but is **not edited** through the current screens.
- **Uses (equipment usage):** The running application tracks a **Status** on usage rows (e.g., In Use, Returned, Reserved) to distinguish active reservations from completed ones. Queries that need “currently using” equipment (see equipment reporting) rely on **active usage semantics** (including `EndDate IS NULL` where applicable in the DAO).
- **Equipment vs. Devices:** The UI treats each **device unit** (row in **Devices**, keyed by **DeviceNumber**) as the object users add, update, or remove. Adding “equipment” in the app **inserts both** an **Equipment** (model/type) row and a linked **Devices** row, matching the Phase 2 separation of model vs. physical unit.

No change was required to the **high-level EER mapping** decided in Phase 2; revisions above are **implementation-level** simplifications or extensions for the application layer.

2. Database Instance Creation

Environment

- **DBMS:** Microsoft SQL Server (2025 image in Docker for local development).
- **Database name:** `ResearchLabManager`.
- **Connectivity:** JDBC URL
`jdbc:sqlserver://localhost:1433;databaseName=ResearchLabManager;encrypt=true;trustServerCertificate=true;`
 with SQL authentication (`sa` user; password configured to match the container).
- GitHub URL: <https://github.com/RoohitD/researchlabmanager>

Provisioning steps (summary)

1. Start SQL Server in Docker (see project `README.md` for the `docker run` example, including `ACCEPT_EULA` and `MSSQL_SA_PASSWORD`).
2. Copy the provided backup `ResearchLabManager.bak` into the container and **restore** it into an instance database named `ResearchLabManager` (standard `RESTORE DATABASE` workflow).
3. Verify connectivity from the host (port **1433**) and from the Java application.

This approach gives a **repeatable** environment for grading and demos: the same backup + container recipe reproduces the schema and baseline data.

3. Data Population

Per Phase 3 requirements, each table must contain **enough rows** to exercise **all relationships** and **all required queries**.

- **Primary approach:** The restored **ResearchLabManager.bak** provides a **seed dataset** with interconnected **LabMembers** (including subclasses), **ResearchProjects**, **WorksOn**, **Grants**, **Equipment / Devices**, **Uses**, **Publications**, **Author**, and **Mentorship** rows so that reporting (top-funded projects, mentorship/publication analytics, student publications by major/year, etc.) returns non-empty, realistic output.
- **Additional inserts (recommended for submission):** Separate **.sql script files** (one per table or one ordered master script) with **INSERT** statements can be included alongside the backup so graders can **re-populate** an empty schema or **add** more tuples without relying on the BAK. When writing those scripts, **respect foreign key order** (e.g., **LabMembers** before **Student/Faculty/Collaborators**; **ResearchProjects** before **Grants** and **WorksOn**; **Equipment** before **Devices**; **Devices** and **LabMembers** before **Uses**; **Publications** before **Author**).

The application may also be used to **add, update, or delete** members, projects, equipment, and usage during a demo to show live CRUD behavior.

4. Application Development

Technology stack

- **Language:** Java
- **UI:** JavaFX (FXML: **ResearchLabManager.fxml**)
- **Database access:** JDBC (**LabManagerDAO** with **PreparedStatement / Statement**)
- **Build:** Maven (**pom.xml**)

Structure

- **LabManagerController:** Handles UI events, dialogs, table refresh, and captures **standard output** for DAO methods that print results (reporting tab).
- **LabManagerDAO:** Encapsulates SQL for CRUD and all required reporting queries.

- **DB.java**: Optional console driver to smoke-test DAO methods without the GUI.

User interface (menu-driven, practical)

The UI is organized into **three tabs**, matching the three required menu areas:

1. **Project and Member Management**
2. **Equipment Usage Tracking**
3. **Grant and Publication Reporting**

Within each tab, **buttons and fields** act as the “menu”: refresh lists, add/update/remove entities, and run the specialized queries. This satisfies the expectation of a **menu-driven** system without investing in decorative GUI work.

5. Mapping to Phase 3 Functional Requirements

5.1 Project and Member Management

Requirement	Implementation
Query, add, update, and remove members	Table view + refresh; dialogs for add/update; delete with confirmation; DAO: <code>listMembers</code> , <code>addMember</code> , <code>updateMember</code> , <code>deleteMember</code> (with cleanup on related <code>WorksOn</code> / <code>Uses</code> where needed).
Query, add, update, and remove projects	Table view + refresh; dialogs for add/update (title, status, member ID list); delete with confirmation; DAO: <code>listProjects</code> , <code>addProject</code> , <code>updateProject</code> , <code>deleteProject</code> , <code>getMemberIdsForProject</code> , <code>setProjectMembers</code> .
Display status of a project	User enters Project ID ; DAO: <code>getProjectStatus</code> .

Requirement	Implementation
Members who worked on projects funded by a given grant	User enters Grant ID ; DAO: <code>getMembersByGrant</code> (joins <code>LabMembers</code> , <code>WorksOn</code> , <code>Grants</code>).
Mentorship among members who worked on the same project	Button runs DAO: <code>printMentorshipByProject</code> (joins <code>Mentorship</code> , <code>LabMembers</code> , <code>WorksOn</code> , <code>ResearchProjects</code> for common <code>ProjectID</code>).

5.2 Equipment Usage Tracking

Requirement	Implementation
Query, add, update, and remove equipment	Table of devices (model/type/status); DAO: <code>listEquipments</code> , <code>addEquipment</code> (insert Equipment + Devices), <code>updateEquipment</code> , <code>deleteEquipment</code> .
Query, add, update, and remove equipment usage	Usage table; DAO: <code>listEquipmentUsages</code> , <code>addEquipmentUsage</code> , <code>updateEquipmentUsage</code> , <code>deleteEquipmentUsage</code> .
Status of a piece of equipment	User enters Device # ; DAO: <code>getDeviceStatus</code> on Devices .
Members currently using a device and projects they work on	User enters Device # ; DAO: <code>showEquipmentUsers</code> (joins <code>Uses</code> , <code>LabMembers</code> , <code>WorksOn</code> , <code>ResearchProjects</code> ; restricts to active usage per query logic, e.g. open <code>EndDate</code>).

5.3 Grant and Publication Reporting

Requirement	Implementation
Top 5 projects by total grant funding (Budget), descending	DAO: <code>getTop5FundedProjects</code> — <code>SUM(Budget)</code> grouped by project, <code>TOP 5</code> ordered by total.
Mentor(s) whose mentees collectively have the largest number of publications	DAO: <code>getTopMentorByMenteePubs</code> — joins <code>Mentorship</code> , <code>LabMembers</code> , <code>Author</code> , <code>TOP 1</code> by count.
Total student publications per major and per publication year	DAO: <code>getStudentPubsByMajorAndYear</code> — <code>Student</code> ⋈ <code>Author</code> ⋈ <code>Publications</code> , grouped by <code>Major</code> and <code>Year</code> .
Given date X , projects ended before X and count of grants per project	Date picker; DAO: <code>getProjectsBeforeDate</code> — filter on <code>EndDate</code> , <code>LEFT JOIN</code> grants for counts.
Three most productive years for student publications	DAO: <code>getTop3ProductiveYears</code> — restrict authors to <code>Student</code> , group by year, <code>TOP 3</code> .

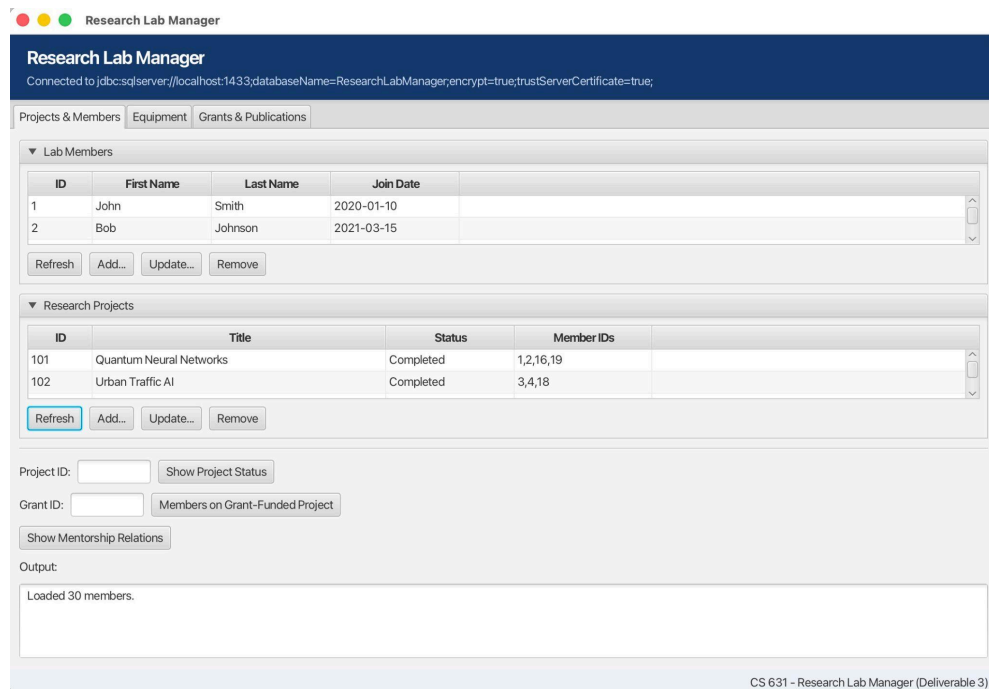
6. Design Justifications

- **DAO + thin controller:** Centralizing SQL in `LabManagerDAO` keeps the UI readable and makes queries testable from `DB.java` or future unit tests.
 - **Equipment add = model + unit:** Creating both **Equipment** and **Devices** on “add equipment” preserves the Phase 2 data model (catalog vs. inventory) while exposing a single workflow to users.
 - **Project members via WorksOn:** Replacing all rows for a project on update is simple and correct for small teams; it avoids duplicate (`MemberID`, `ProjectID`) pairs consistent with the Phase 2 uniqueness intent.
 - **Stdout capture for reporting:** Some analytics methods print tabular output; the controller temporarily redirects `System.out` into a `TextArea` so reporting stays consistent without rewriting every query as a string-returning API.
-

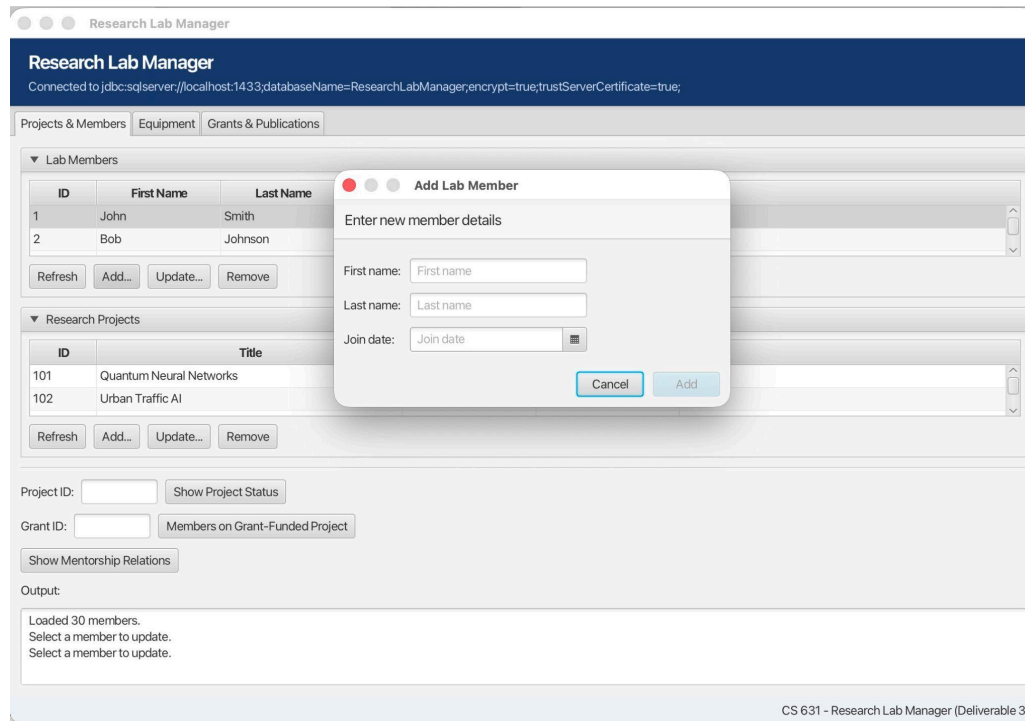
7. Challenges Encountered

- **Foreign key ordering:** Whether restoring from backup or running **INSERT** scripts, table order matters (parents before children). Mistakes show up quickly as FK violations; the team documented a safe insertion order for manual scripts.
 - **SQL Server dialect vs. lecture MySQL examples:** Features such as **TOP** and **STRING_AGG** are used where appropriate; project listings aggregate member IDs for display in one round trip.
 - **“Current” equipment use:** Defining “currently using” required aligning **usage status** and **EndDate** semantics in **Uses** with the reporting query so results match demo expectations.
 - **Schema richness vs. UI scope:** Phase 2 DDL includes more columns than the minimal forms (e.g., **WeeklyHours** on **WorksOn**). The team **documented** those gaps here rather than over-expanding the GUI, per course guidance to keep the interface practical.
-

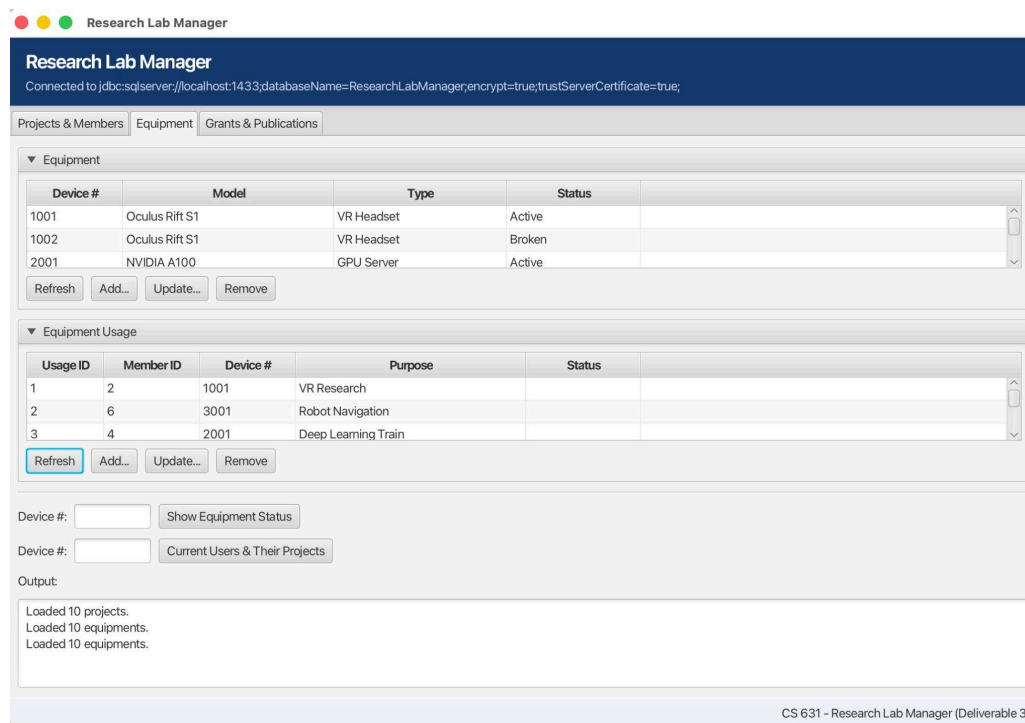
8. Application Screenshots



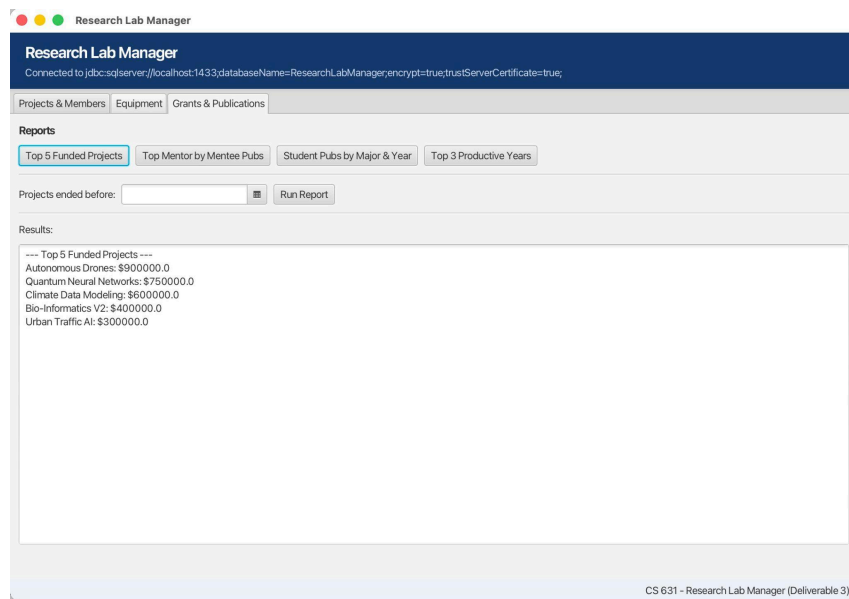
Projects & Members Screen



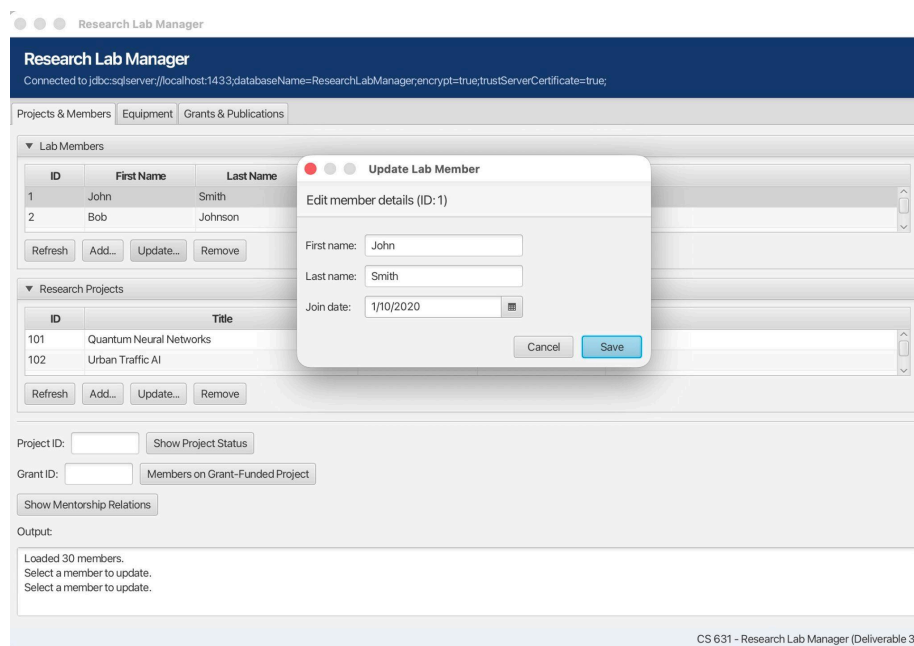
Add Lab Member, Projects, Equipment, Equipment Usage Screen



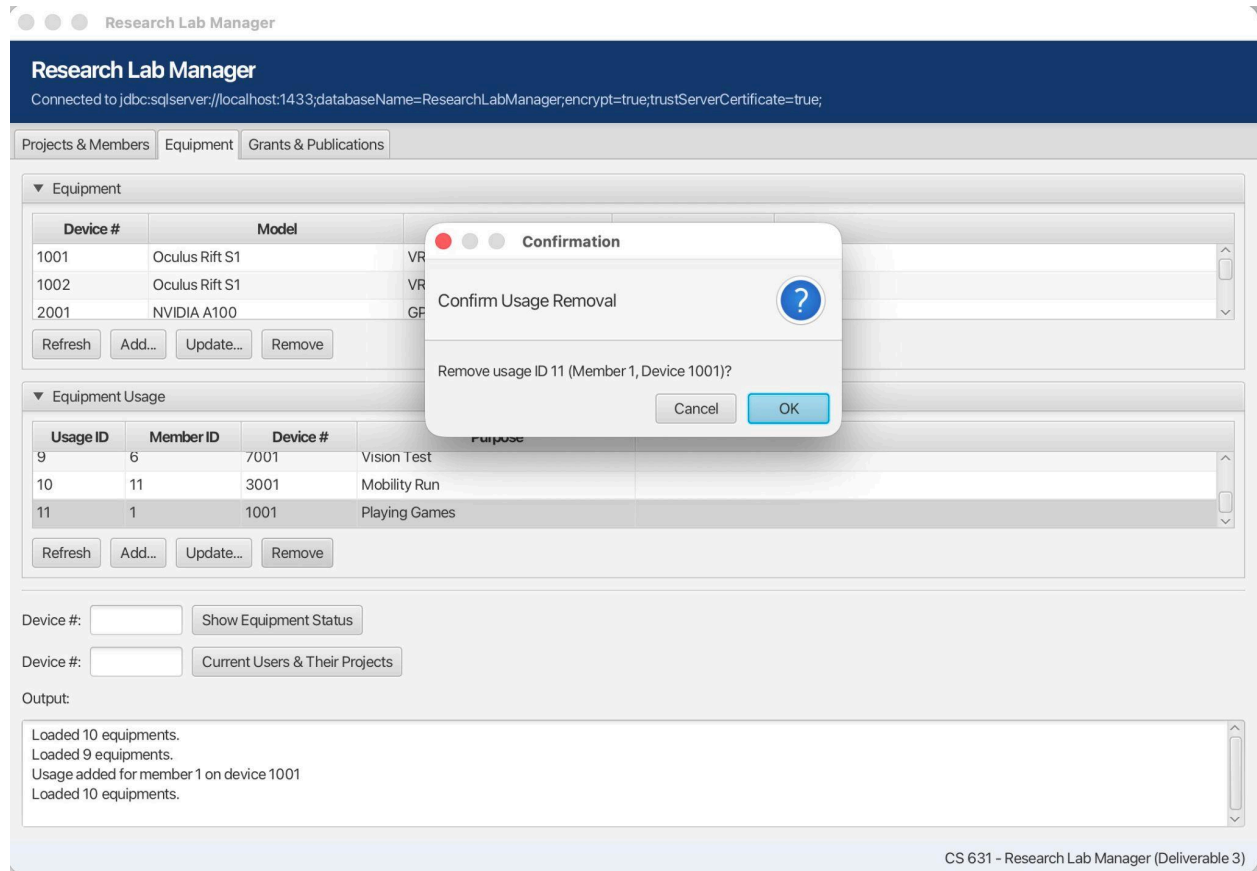
Equipment and Equipment Usage screen



Grants and Publications Screen



Update Lab Member, Projects, Equipment & EquipmentUsage Screen



Screen for removing item from lab members, projects, equipment and equipmentUsage list
